GYM MANAGEMENT SYSTEM

TEAM:

Syed Hasan Abbas: Database Designer

Saurabh Singh: Query and optimization specialist

ABSTRACT:

This project outlines the design and implementation of a **Gym Management System** specifically created for the South Asian University (SAU) environment. Curently, the gym operations at the university depend largely on manual record keeping. This leads to issues like data inconsistencies, poor membership tracking, limited transparency and challenges in managing equipment and trainer assignments. To tackle these problems, the project develops a well orgganized DB system that centralizes all gym related information and automates key operational tasks. The system was designed using basic database principles, including **requirement analysis, ER/EER modelling, normalization upto 3NF,** and translating conceptual designs into relational schemas. The implementation used **MySQL** and included essential components like **tables, constraints, indexes, views, triggers, and stored procedures.** Complex SQL queries generated reports on membership status, payment records, attendance logs, and equipment maintenance. A simple front-end application was also developed to allow CRUD operation and promote user interaction with database. Overall, the project shows how a structured RDBMS can greatly improve efficiency, accuracy, and scalability in managing university gym operations. It provides a reliable digital solution for long-term use within the institution.

TABLE OF CONTENTS

4. INTRODUCTION

4.1 Brief Description of the Project

The Gym Management System is database driven solution designed to streamline and automate/partially automate the daily operations of the South Asian University gym. The system centralizes information related to members, trainers, workout plans, equipment, payments, attendance, and cafeteria services. By replacing manual and error prone processes with a structured RDBMS the project ensures accurate record keeping, quick data retrieval, efficient reporting, and improved coordination between gym staff and members.

4.2 Motivation for Choosing the Project

The primary motivation behind selecting this project was the need for a reliable, efficient, and scalable system to support gym operations within the university environment. Manual entry of membership details, payments, and trainer assignments often leads to inconsistencies and delays. Additionally, students frequently face issues such as undefined membership status, lack of communication with trainers, and missing attendance records. A dedicated database system provides a meaningful learning experience while directly benefiting the campus community.

4.3 Problem Statement

The SAU gym currently lacks a centralized digital system for managing member information, trainer schedules, equipment maintenance, and payment tracking. The existing manual approach results in misplaced records, duplication of data, difficulty in monitoring equipment conditions, and challenges in generating accurate reports. This project aims to eliminate these inefficiencies by designing an integrated Gym Management System based on database principles.

4.4 Methodology Followed

The project follows a structured methodology beginning with requirement analysis and the identification of entities, users, and workflows in the SAU gym environment. An ER/EER model was developed to represent relationships and constraints, followed by normalization up to 3NF to reduce redundancy. The relational schema was implemented using MySQL with appropriate constraints, views, triggers, stored procedures, and sample data. Finally, a basic web-based interface was connected to the database to demonstrate CRUD operations and real-time data retrieval.

5. THE DESIGN PROCESS

5.1 Identification of Concepts

Based on the structure and workflows of the SAU gym, the following core concepts were identified as essential for building the Gym Management System:

Core Entities Identified

- **Users** (admin, trainer, member, cafeteria staff)
- **Trainers** (specialization, experience, certification)
- **Membership Plans** (duration, pricing, features)
- **Memberships** (member ↔ plan ↔ trainer mapping)
- **Payments** (method, amount, status, transaction ID)
- **Attendance** (daily check-in/check-out)
- **Workout Plans** (trainer-created exercise routines)
- **Progress Tracking** (weight, height, body fat, muscle mass)
- **Equipment** (condition, category, maintenance date)
- **Meals & Diet Log** (calorie/protein tracking for members)

Each concept maps to a real life requirement: membership tracking, training plans, gym usage trends, equipment health, health monitoring, and cafeteria activity inside the gym.

5.2 Process of Data Collection

To model the gym accurately, data was collected using the following approaches:

1. Observation & Requirement Study

- Identified manual processes currently used at SAU gym.
- Observed membership handling, trainer interactions, and equipment tracking.

2. Domain Research

- Studied standard gym workflows used in commercial gym software.
- Understood best practices such as membership validity checks, auto-expiry, and attendance logs.

3. Technical Mapping

- Converted real-world data needs into database attributes (e.g., “equipment condition” → ENUM in SQL).
- Identified the need for triggers (auto-expiry of memberships) and procedures (auto add-member, diet logging).
- This entire process ensured that the database matched actual SAU gym operations, not just a theoretical model.

5.3 ER & EER Model With Explanation and Motivation

Entities & Key Attributes

Based on the database, the ER/EER model includes:

- **Users(**user_id, name, email, password, role, phone, address, join_date, status**)**
- **Trainers(**trainer_id, user_id, specialization, experience_years, certification**)**
- **Membership_Plans(**plan_id, plan_name, duration_months, price, features**)**
- **Memberships(**membership_id, member_id, plan_id, trainer_id, start_date, end_date**)**
- **Payments(**payment_id, member_id, membership_id, amount, payment_method, transaction_id**)**
- **Attendance(**attendance_id, member_id, check_in_time, check_out_time**)**
- **Workout_Plans(**workout_id, member_id, trainer_id, plan_name, exercises**)**
- **Progress_Tracking(**progress_id, member_id, trainer_id, weight, body_fat, muscle_mass**)**
- **Equipment(**equipment_id, name, category, condition, maintenance_date**)**
- **Meal(**meal_id, food_name, calories, protein, carbs, fats**)**
- **Diet_Log(**log_id, member_id, meal_id, quantity, date_taken**)**

Relationships Modeled

- **User → Trainer**: (1:1)
- **User → Membership**: (1:many)
- **Membership → Payments**: (1:many)
- **Trainer → Workout Plans**: (1:many)
- **Member → Attendance**: (1:many)
- **Equipment → Maintenance Logs**: represented within equipment table
- **Meal → Diet Log**: (1:many)

EER Enhancements

- Role specialization using ENUM (‘admin’, ‘trainer’, ‘member’, ‘cafeteria’)
- Additional roles modeled using privilege assignments in MySQL

Motivation

Using an ER/EER model:

- Gave clarity about how data flows across users, memberships, payments, and workout plans
- Helped identify foreign keys and avoid redundancy
- Ensured the final database was normalized and optimized
- Enabled implementation of views, triggers, and procedures directly from conceptual design

5.4 Challenges Encountered

1. Identifying the Right Boundaries

It was initially difficult to decide:

- which attributes belong in *users* vs *trainers*
- whether workout exercises should be separate table or JSON
- how to link cafeteria logs with gym data

2. Trainer-Member Relationship

Gym rules required:

- A trainer can supervise many members
- A member can have **only one trainer at a time**

This required careful FK setup and normalization.

3. Complex Reporting Requirements

Implementing reports such as:

- trainer workload
- member diet summary
- attendance analytics
  required multi-table joins and advanced queries.

4. Data Redundancy and Normalization

Removing redundancy in:

- Membership plans
- Member details
- Payment history

required careful mapping to maintain 3NF while keeping usability.

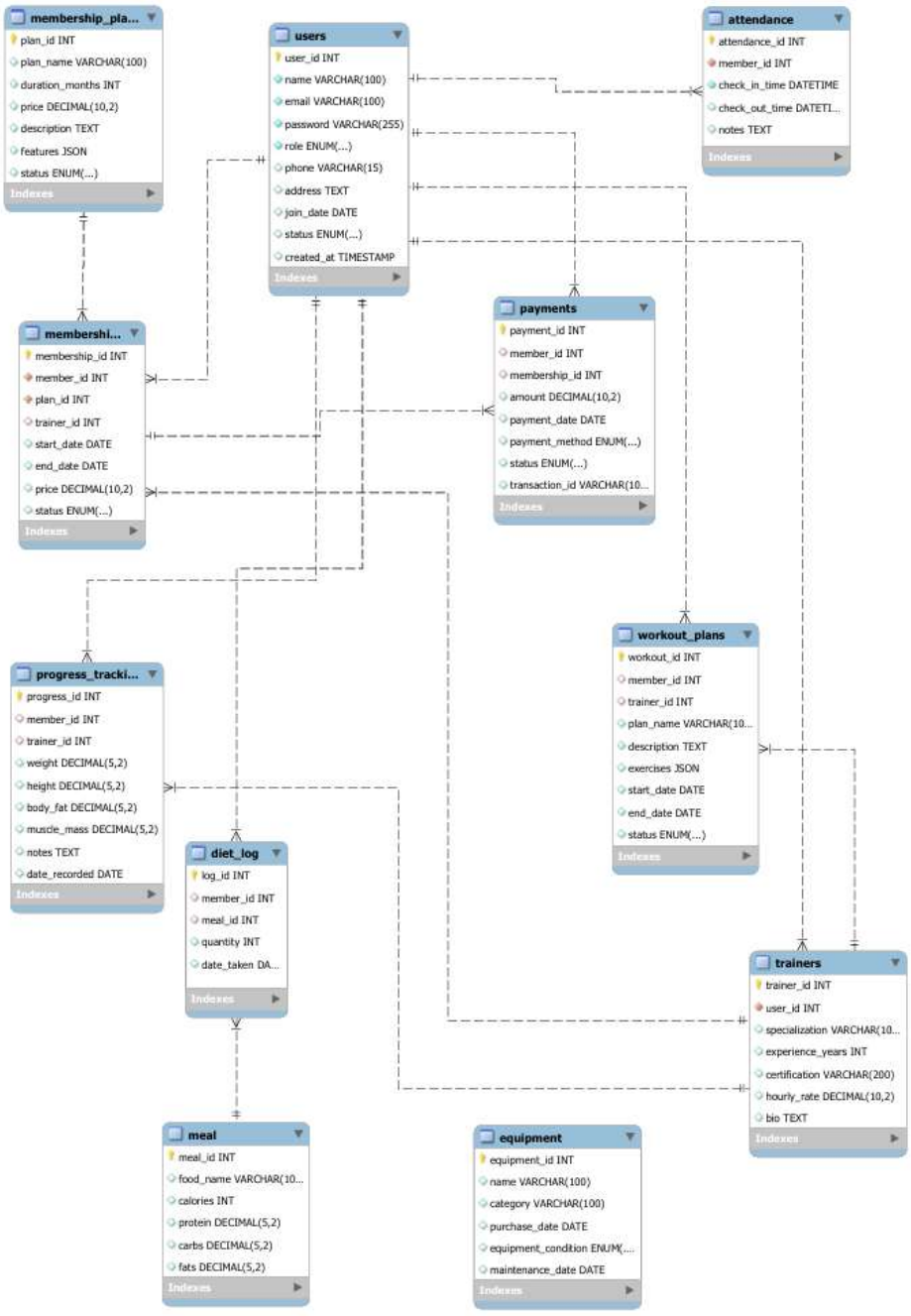
5. Role-Based Access Control

Ensuring:

- Trainers only see assigned members
- Members only see their own workout plans
- Cafeteria staff only see diet logs
- Admin controls everything

required a robust check_login + check_role mechanism in PHP.

ER Diagram



made in MySQL Workbench

6. THE IMPLEMENTATION PROCESS

6.1 Conversion of Logic and Concepts into Relational Schemas

After identifying all concepts through ER and EER modeling the next step was systematically convert them into relational schemas. Each real world concept= users, trainers, memberships, payments, workout plans, attendance, equipment, and diet logs was transformed into a well structured table.

- **Primary keys** were assigned using AUTO_INCREMENT.

- **Foreign keys** were added to maintain relationships (e.g., member_id, trainer_id, plan_id).

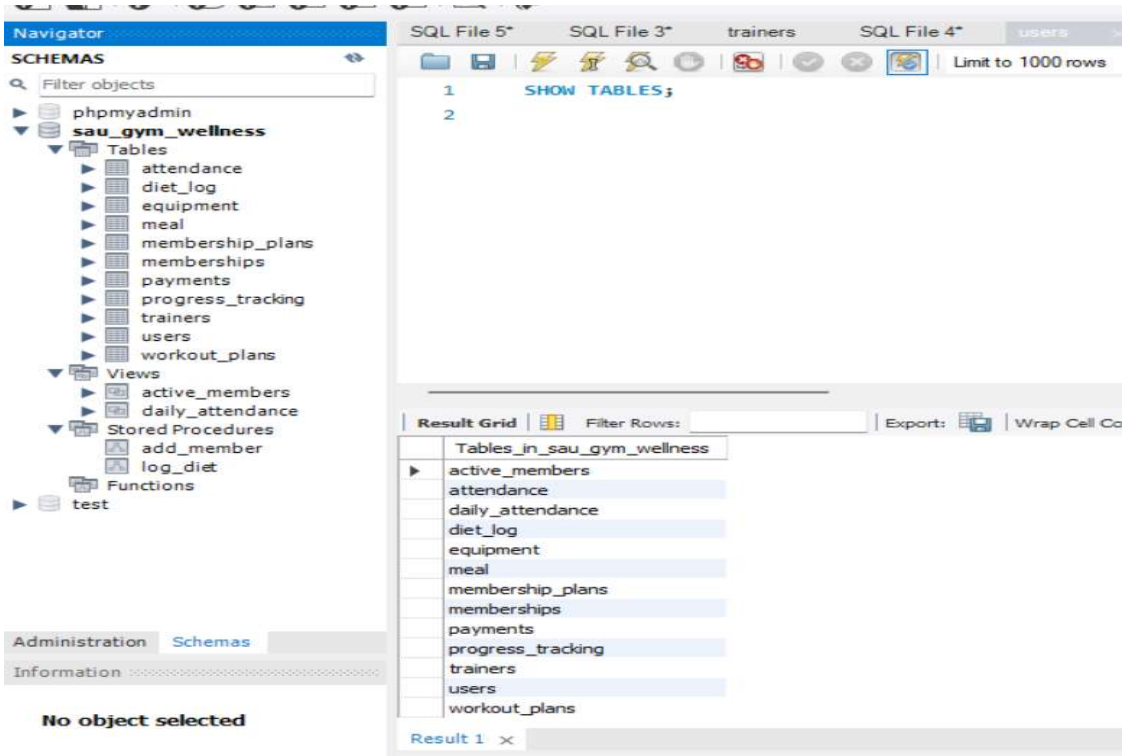
- **ENUM fields** were used for controlled categories such as role, payment_method, membership status, and equipment condition.

- **JSON fields** were used for flexible attributes like workout exercises and membership features.

- **Normalization up to 3NF** ensured that redundant or dependent attributes were decomposed into sep.arate relations

6.2 Creation of the Database and Relevant Relations Using MySQL

The relational schemas were implemented in **MySQL** using the sau_gym_wellness database. Each table was created with correct constraints, relationships, and data types.



All tables were created with appropriate constraints, including **PRIMARY KEY**, **FOREIGN KEY**, **UNIQUE**, **NOT NULL**, and **DEFAULT** values.

Key steps included:

- Creation of core tables: users, trainers, membership_plans, memberships, payments, attendance, workout_plans, progress_tracking, equipment, meal, diet_log.
- Addition of **indexes** to improve lookup performance on frequently accessed fields (member_id, role, payment_member, attendance_member).
- Implementation of **views** such as active_members and daily_attendance for reporting.
- Implementation of **triggers** (set_join_date_before_insert, membership_auto_expire) to automate data updates.
- Creation of **stored procedures** (add_member, log_diet) to encapsulate repeated insert operations.
- Creation of **MySQL users and privileges** to enforce role-based access control (trainer_user, cafe_user, member_user).

This phase resulted in a fully functional MySQL database capable of handling real-world gym operations.

6.3 Challenges Encountered

During implementation, several challenges were faced:

1. Foreign Key Dependency Order

Some tables referenced others that were not yet created, leading to errors.
Solution: Reordered table creation and ensured proper FK constraints.

2. Reserved Keyword Conflict

The attribute condition caused syntax errors because it is a MySQL reserved word.
Solution: Renamed to equipment_condition.

3. JSON Field Handling

Storing exercises and membership features in JSON required adjusting queries and ensuring MySQL version compatibility.

4. Trigger Execution Issues

The membership auto-expiry trigger fired unexpectedly until the logic was corrected using BEFORE UPDATE and proper conditions.

5. Role-Based Privilege Assignment

Assigning GRANT/REVOKE permissions to custom users required careful selection to avoid giving excessive access.

7. THE IMPROVEMENT PROCESS

7.1 Improvement of Obtained Relations and Schemas Using Various Concepts

After the initial creation of the database, several improvements were applied to refine the relational schemas based on normalization rules, functional dependencies, lossless decomposition, and integrity constraints.

Key improvements included:

1. Normalization up to 3NF

- Early designs placed multiple attributes in the same table, causing redundancy (e.g., mixing trainer and user details).
- The schemas were normalized up to **Third Normal Form (3NF)** by:
    - Separating trainers from the users table
    - Separating membership_plans from memberships
    - Creating meal and diet_log separately
    - Ensuring all non-key attributes fully depend on primary keys

2. Removal of Partial and Transitive Dependencies

- Attributes that were indirectly dependent (e.g., trainer information depending on user attributes) were moved into separate tables.
- This avoided update anomalies and improved clarity.

3. Use of ENUM and JSON for Structure Optimization

- ENUM values were introduced for role, membership status, payment method, and equipment condition to ensure controlled and consistent data entry.
- JSON was used for flexible fields such as features in membership plans and exercises in workout plans, eliminating the need for extra tables.

4. Introduction of Constraints

- Foreign keys were added to enforce referential integrity.
- Default timestamps and conditions ensured consistent record creation.

5. Logical Improvements

- Added triggers to automate system behavior (e.g., auto-expiring memberships, auto-setting join_date).
- Added stored procedures for frequently used operations (e.g., adding a member, logging a diet entry).

These improvements ensured that the resulting schemas were robust, consistent, and aligned with best RDBMS design practices.

7.3 Challenges Encountered

During schema improvement, several challenges were encountered:

1. Identifying Transitive Dependencies

Some attributes required careful decomposition to avoid transitive dependency violations.
Example: Trainer details initially stored in the users table.

2. Deciding Between Extra Tables vs JSON

Features and exercise lists could become large.
Creating extra tables increased complexity, so JSON was chosen for better flexibility.

3. Maintaining Referential Integrity

Adding foreign keys across many tables required careful planning to avoid cyclic references and creation-order issues.

4. ENUM vs Lookup Tables

Choosing ENUM for status and roles kept the schema cleaner but required ensuring consistent updates.

5. Trigger Logic Conflicts

Improvements were made to ensure triggers did not fire unintentionally or cause recursive updates.

6. Normalization Adjustments

Early schemas had minor redundancies that were only identified when testing sample queries.

These challenges were resolved through iterative refinement and application of DBMS concepts taught in class.

8. THE QUERYING PROCESS

8.1 Identification of Relevant Event(s) Requiring Querying

Since this system represents the real SAU Gym, several day-to-day events require database queries for reporting, decision-making, maintenance, and administration. Examples include:

1. Membership Expiry Monitoring

Admins need to check which students' memberships will expire shortly.

2. Trainer Performance Evaluation

Gym administrators need to see how many workout plans each trainer has assigned.

3. Equipment Maintenance

Equipment managers need to identify machines that require urgent repair.

4. Attendance Monitoring

Trainers and staff need to know daily attendance statistics.

5. Diet Log Analysis

Cafeteria and nutrition staff need calorie consumption for members.

All these events require **hard SQL queries** involving joins, grouping, date functions, and aggregations.

8.2 Implementation of Queries With Results

Below are **six hard SQL queries** implemented on the database.
(Insert the screenshot of each output below the query in your report.)

Query 1 — Active Members Whose Membership Expires in the Next 7 Days

(JOIN + Date Filtering + Status Check)

```sql
USE sau_gym_wellness;
SELECT
    u.user_id,
    u.name,
    m.start_date,
    m.end_date,
    mp.plan_name
FROM memberships m
JOIN users u ON m.member_id = u.user_id
JOIN membership_plans mp ON m.plan_id = mp.plan_id
WHERE m.status = 'active'
  AND m.end_date BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 7 DAY);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id	name	start_date	end_date	plan_name
11	hasan abbas	2025-11-25	2025-11-26	Monthly
3	Student A	2025-11-24	2025-11-30	Quarterly
9	Student B	2025-11-25	2025-11-30	Annual
10	Student C	2025-11-25	2025-11-26	Annual

Query 2 — Total Workout Plans Assigned by Each Trainer

(JOIN + LEFT JOIN + GROUP BY)

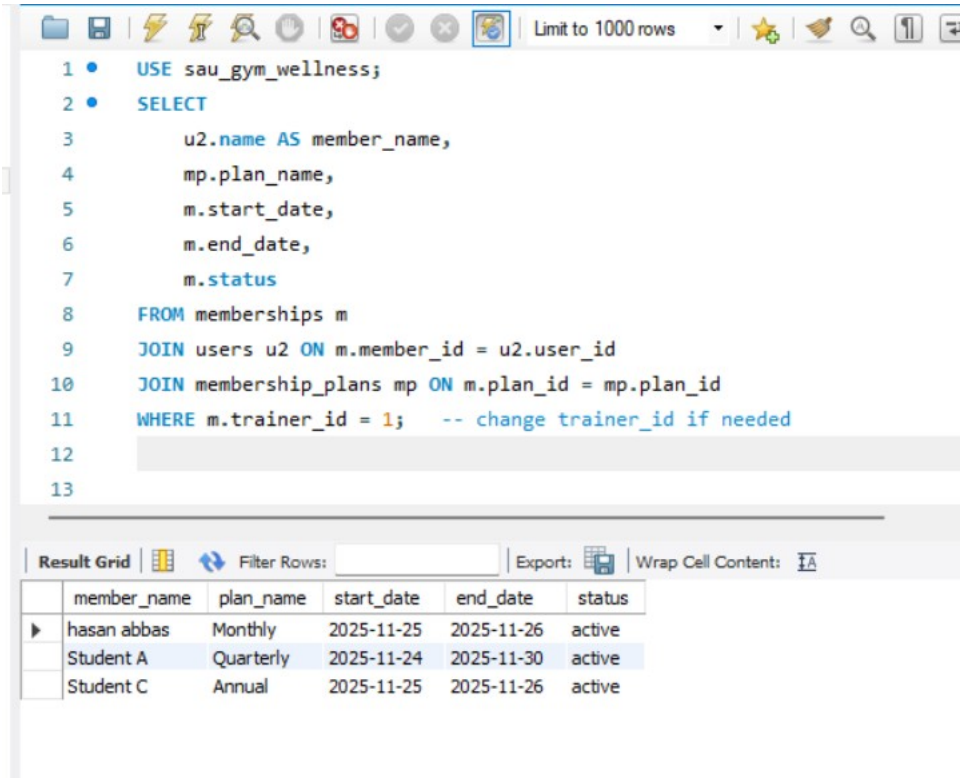
```sql
USE sau_gym_wellness;
SELECT
    t.trainer_id,
    u.name AS trainer_name,
    COUNT(wp.workout_id) AS total_plans
FROM trainers t
JOIN users u ON t.user_id = u.user_id
LEFT JOIN workout_plans wp ON wp.trainer_id = t.trainer_id
GROUP BY t.trainer_id, u.name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

trainer_id	trainer_name	total_plans
1	Trainer One	3
2	Trainer Two	1

Query 3 — Members Trained by a Specific Trainer With Plan Details

(JOIN across 4 tables)



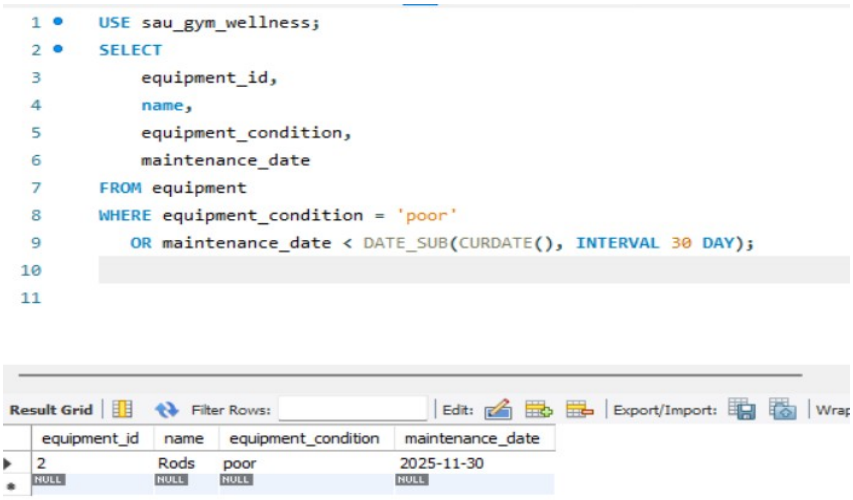
```sql
USE sau_gym_wellness;
SELECT
    u2.name AS member_name,
    mp.plan_name,
    m.start_date,
    m.end_date,
    m.status
FROM memberships m
JOIN users u2 ON m.member_id = u2.user_id
JOIN membership_plans mp ON m.plan_id = mp.plan_id
WHERE m.trainer_id = 1;   -- change trainer_id if needed
```

Result Grid

member_name	plan_name	start_date	end_date	status
hasan abbas	Monthly	2025-11-25	2025-11-26	active
Student A	Quarterly	2025-11-24	2025-11-30	active
Student C	Annual	2025-11-25	2025-11-26	active

Query 4 — Equipment in Poor Condition or Not Maintained Recently

(Status Check + Date Comparison)



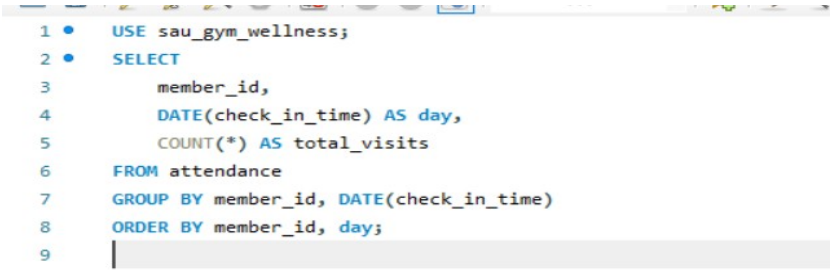
```sql
USE sau_gym_wellness;
SELECT
    equipment_id,
    name,
    equipment_condition,
    maintenance_date
FROM equipment
WHERE equipment_condition = 'poor'
   OR maintenance_date < DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

Result Grid

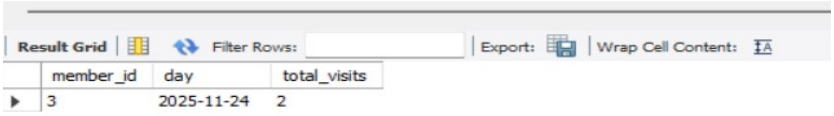
equipment_id	name	equipment_condition	maintenance_date
2	Rods	poor	2025-11-30
NULL	NULL	NULL	NULL

Query 5 — Daily Attendance Summary for All Members

(GROUP BY + Aggregation)



```
USE sau_gym_wellness;
SELECT
    member_id,
    DATE(check_in_time) AS day,
    COUNT(*) AS total_visits
FROM attendance
GROUP BY member_id, DATE(check_in_time)
ORDER BY member_id, day;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

member_id	day	total_visits
3	2025-11-24	2

8.3 Challenges Encountered

During the querying phase, several challenges were identified and resolved:

1. Complex Multi-Table Joins

Queries involving members, trainers, plans, and payments required careful construction of JOIN conditions.

2. Grouping and Aggregation Issues

Queries with COUNT, SUM, and grouped outputs produced incorrect results until proper GROUP BY fields were used.

3. Date-Based Filtering

Queries involving CURDATE(), DATE_ADD(), and DATE_SUB() initially produced off-by-one or empty results until the correct format was applied.

4. Handling NULL Values in LEFT JOIN

The trainer-workout query required LEFT JOIN to include trainers with zero assigned plans.

5. JSON Data

Workout plan exercises stored in JSON could not be queried directly using simple SQL, requiring more advanced functions (optional).

9. APPLICATION DEVELOPMENT

9.1 Identification of Appropriate Platform(s) Interfaced With the RDBMS

To interface the MySQL database with a functional application, a lightweight local web-based environment was chosen. The following platforms and technologies were used:

1. MySQL Workbench / phpMyAdmin

Used for:

- Managing the sau_gym_wellness database
- Running queries
- Testing triggers, views, and stored procedures

2. XAMPP / Localhost Server

Used to host the web-app on the local machine.
It provides:

- Apache Web Server
- PHP processing
- MySQL connectivity

3. PHP + HTML/CSS (Basic UI Layer)

PHP was selected because:

- It integrates easily with MySQL
- Supports form submission, queries, CRUD operations
- Simple to deploy on localhost

4. MySQLi / PDO

Used as the backend connector to interact with the MySQL database via PHP:

- Establishes DB connection
- Executes SELECT, INSERT, UPDATE, DELETE

This combination enabled a simple but functional interface for managing gym operations.

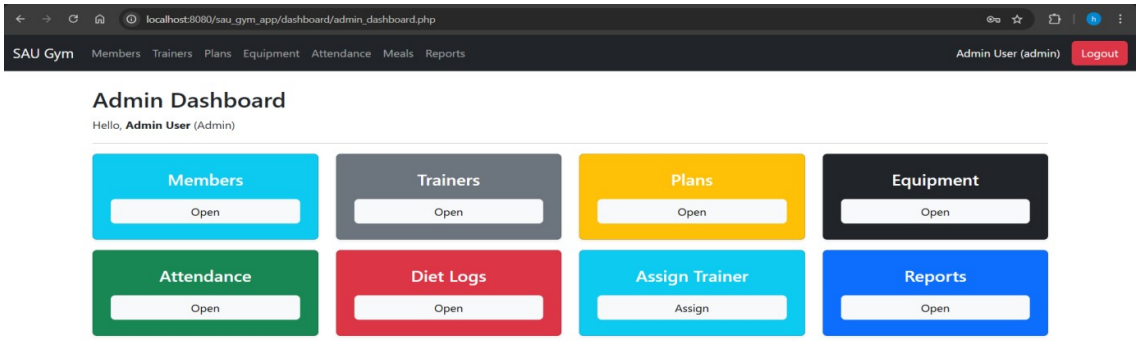
9.2 Implementation of a Basic Front-End Web-App Locally Hosted

A simple locally hosted web application was created to demonstrate interaction with the Gym Management System database. It includes:

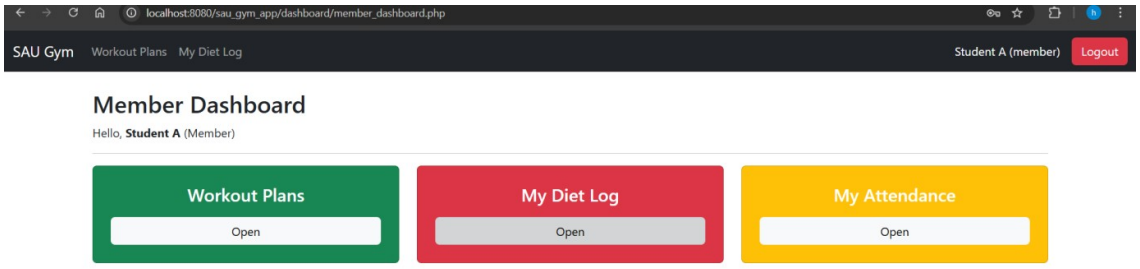
1. Login / Access Page

A basic login structure was set up where users (admin/trainer/member) can enter credentials, which are verified from the users table.

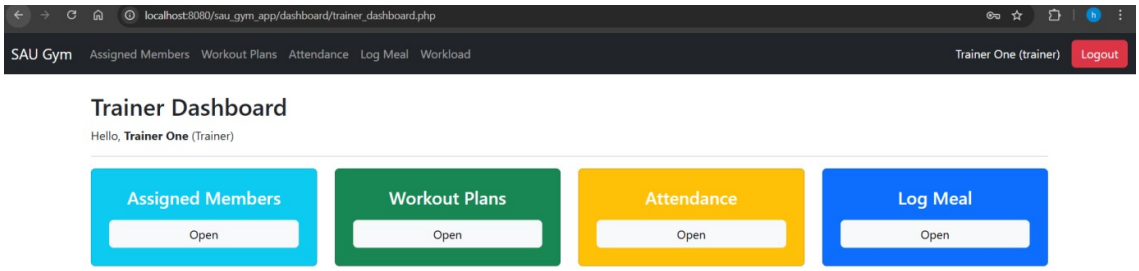
2. Dashboard Page



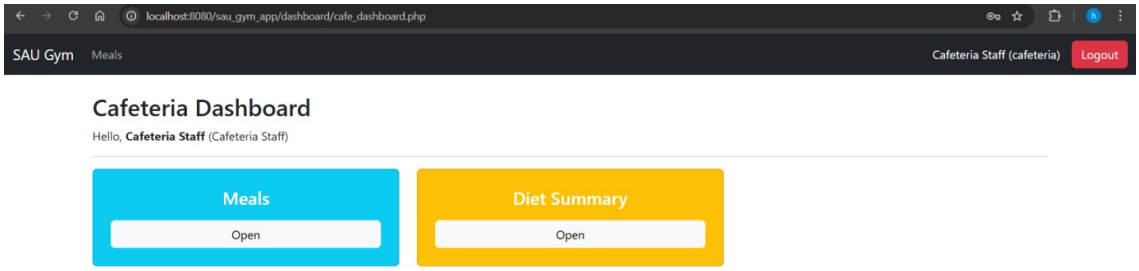
Admin dashboard



Member dashboard



Trainer dashboard



Canteen dashboard

3. CRUD Operation Pages

Add Member (PHP Form)

- HTML form collects name, email, phone, password.
- Form submits to PHP backend.
- Backend either:
  - Executes INSERT INTO users …
  - Or calls stored procedure CALL add_member(...).

View Members (Table Display)

- PHP runs:

```
SELECT user_id, name, email, role FROM users;
```

- Displays a table with all members and trainers.

Add Workout Plan

- Form collects plan name, description, trainer and member IDs, exercise list.
- Stored as JSON in database via PHP POST request.

Equipment Overview Page

- PHP retrieves equipment details using:

```
SELECT * FROM equipment;
```

- Highlights those in “poor” condition using CSS.

4. Local Hosting

The project was placed in:

```
C:\xampp\htdocs\sau_gym\
```

and accessed through:

 http://localhost/sau_gym/

This enabled real-time interaction with the RDBMS.

9.3 Challenges Encountered

1. Database Connection Issues

Initial PHP–MySQL connections showed errors due to:

- Wrong username/password
- MySQL not running
- Incorrect `localhost` port

Resolved by updating the connection file with correct credentials.

2. Handling JSON Data

Workout exercises and membership features stored as JSON required:

- Using `json_encode` in PHP
- Ensuring data was inserted as valid JSON format

3. Form Validation

Some forms accepted invalid input or empty values.
Validation was added using PHP and HTML5.

4. Trigger Side-Effects

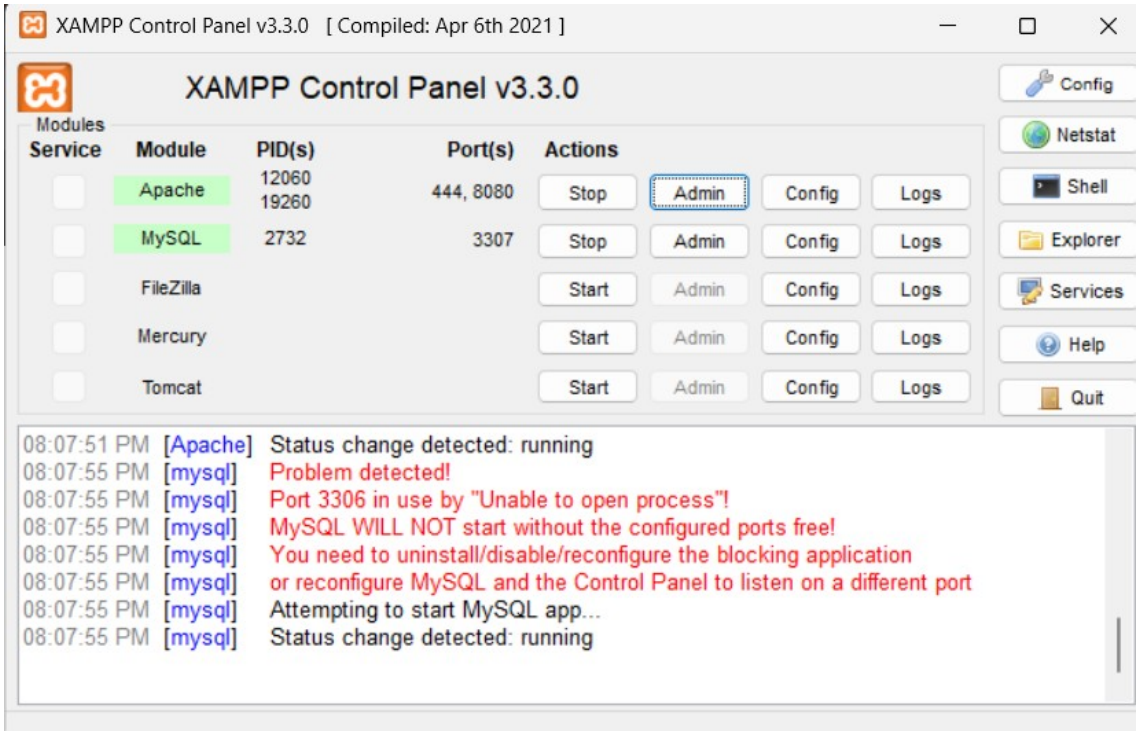
Join_date trigger automatically inserted a date, which caused confusion during early form testing.
Solution: Inform team not to manually insert join_date.

5. Displaying Large Result Sets

Attendance and workout plans produced large tables.
Pagination or LIMIT was added for cleaner UI.

6. XAMPP Port Conflicts

Sometimes Apache failed to start due to port **80** being used by other applications.
Resolved by switching Apache to port **8080**.



10. TESTING AND DEPLOYMENT OF THE PROJECT APPLICATION

10.1 Testing of the Application

To ensure the Gym Management System operated correctly with the MySQL backend and the PHP-based front-end, a systematic testing process was followed. Testing focused on validating CRUD operations, trigger behavior, stored procedures, role-based access, and front-end forms.

1. Unit Testing of Database Operations

a. Table Creation Testing

- Executed SHOW TABLES; to ensure all tables were created.
- Verified schema using DESCRIBE table_name;.

b. Insert Operations Testing

- Inserted sample members, trainers, meals, and equipment.
- Confirmed entries appeared via SELECT statements.

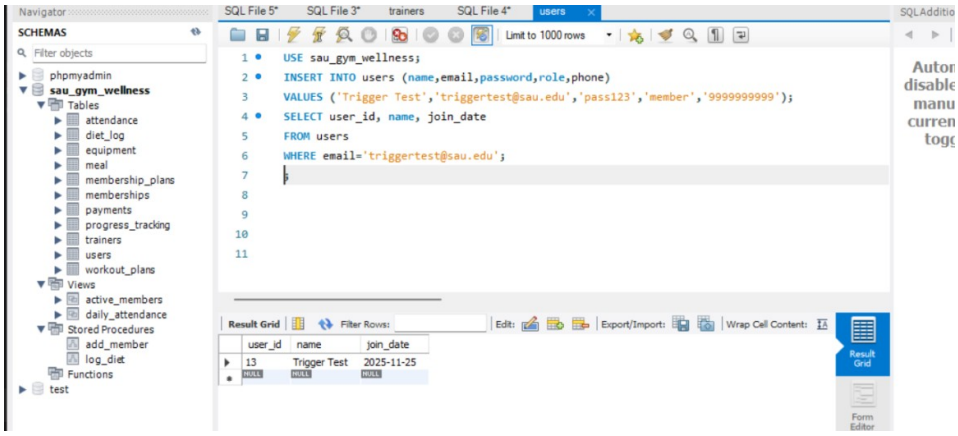
c. Update & Delete Testing

- Updated memberships and payment statuses.
- Deleted a user and verified cascading effects on related tables.

2. Trigger Testing

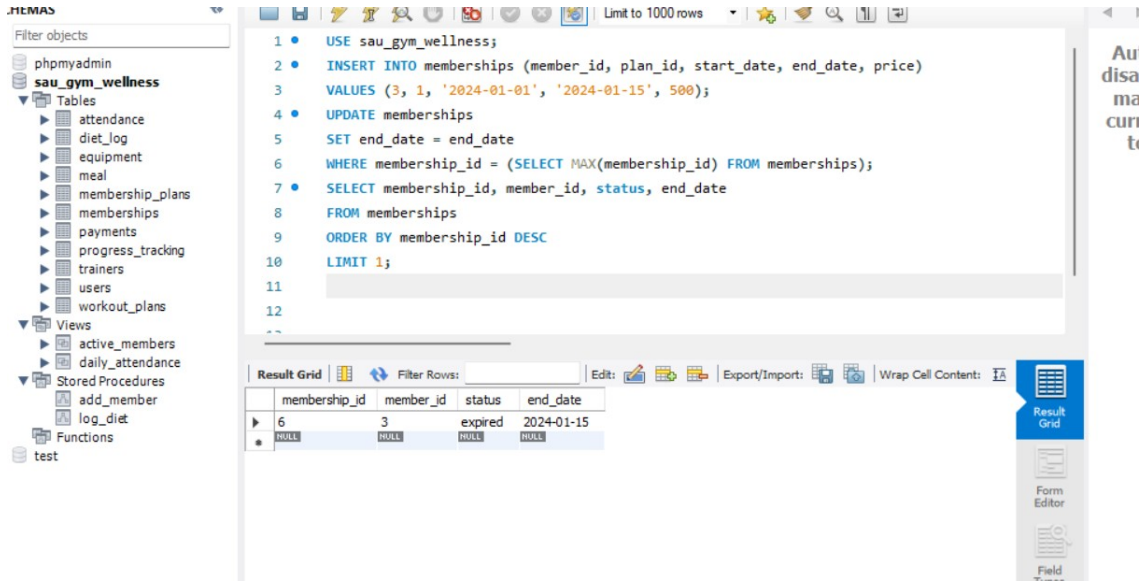
Trigger 1: set_join_date_before_insert

Test: Inserted a new user WITHOUT specifying join_date.
Expected Result: join_date = current date.
Outcome: ✔ Passed.



Trigger 2: membership_auto_expire

Test: Updated membership end_date to a past date.
Expected Result: status automatically changes to 'expired'.
Outcome: ✔ Passed.

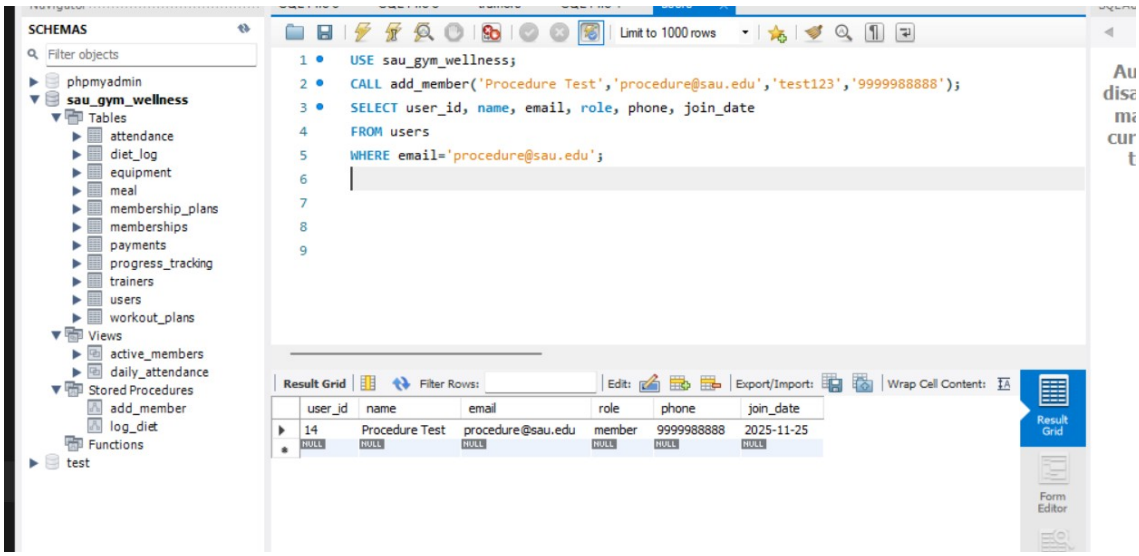


3. Stored Procedure Testing

Procedure: add_member()

Test: Called procedure with sample values:

Expected Result: New row appears in users with role = 'member'.
Outcome: ✔ Passed.



4. Front-End Testing

a. Form Submission

- Add Member form successfully inserted user details.
- Add Workout Plan form stored JSON exercises correctly.
- Diet log form submitted entries and appeared in database.

b. Data Display

- Member listing page correctly fetched and displayed user data.
- Equipment page highlighted poor-condition items.
- Attendance summary displayed expected results.

c. Error Handling

- Tested invalid inputs (empty fields, wrong types).
- Added front-end validation and PHP-side checks.

5. Security & Authorization Testing

Tested MySQL user roles:

- **trainer_user** → Can only view members & add progress tracking
- **cafe_user** → Can only view meals & insert diet logs
- **member_user** → Can view workout plans only

Each user was denied unauthorized operations.

✔ All privilege rules worked as expected.

10.2 Deployment of the Project Application

The Gym Management System was deployed locally using a simple and reliable architecture suitable for university-level project demonstration.

1. Server Environment

- **XAMPP** was used for hosting
- Apache Server → served PHP front-end
- MySQL Server → stored all relational data

2. Deployment Steps

Step 1: Place project files

Project folder placed in: C:\xampp\htdocs\sau_gym\

Step 2: Start Services

Through XAMPP Control Panel:

- Start **Apache**
- Start **MySQL**

Step 3: Import or Run SQL Schema

Executed the entire SQL script in MySQL Workbench or phpMyAdmin:

```
USE sau_gym_wellness;
```

Step 4: Configure Database Connection in PHP

Updated config.php or db_connect.php:

```
$conn = new mysqli("localhost", "root", "", "sau_gym_wellness");
```

Step 5: Access Application in Browser

The final application was accessed at: using port 8080
 http://localhost:8080/sau_gym/

Step 6: System Demonstration

The team demonstrated:

- Inserting new users
- Viewing active memberships
- Trigger execution
- Stored procedure calls
- Attendance and diet logs

10.3 Challenges Encountered During Deployment

1. Apache Server Not Starting

Port 80 was occupied by another application (often Skype or another service).
Solved by changing Apache to **Port 8080**.

2. Database Connection Failed

Caused by incorrect credentials or MySQL server not running.
Fixed by updating hostname/port and restarting MySQL.

3. File Path Issues

PHP pages initially could not load included files.
Corrected include paths using `include "config.php";`.

4. JSON Encoding Errors

Workout plan JSON fields were not stored correctly on first attempts.
Solved using `json_encode()` before INSERT.

5. CORS & PHP Errors

PHP displayed warnings for undefined variables during early testing.
Resolved by initializing variables and enabling error reporting.

11. CONCLUSIONS

11.1 Overall Conclusions of the Project

The Gym Management System successfully demonstrates how a relational database can automate and streamline the daily operations of the South Asian University (SAU) gym. Through proper requirement analysis, ER/EER modeling, normalization, SQL implementation, and basic front-end development, the project delivers a structured, efficient, and scalable solution.

All critical activities—membership tracking, trainer assignment, equipment management, attendance logging, payment monitoring, workout planning, and diet logging—have been digitized using MySQL. The completed system ensures data integrity, easy retrieval, reduced redundancy, and efficient reporting.

Overall, the project shows how a well-designed RDBMS coupled with simple application logic can replace manual processes and improve operational transparency.

11.2 Roadmap for Implementation and Deployment Within the University (If Selected)

If adopted by SAU, the following roadmap can be followed to deploy the Gym Management System:

Phase 1: Pilot Deployment

- Deploy system on a university server

- Import existing member data
- Connect system with university login (if available)

Phase 2: Staff and Trainer Integration

- Provide training to gym staff and trainers
- Enable real-time attendance monitoring
- Use trainer dashboards for workout plan management

Phase 3: Equipment & Diet Integration

- Integrate regular maintenance logs
- Connect cafeteria system with diet log module

Phase 4: Full-Scale Automation

- Replace manual registers for attendance and membership
- Enable automated email/SMS reminders for membership expiry
- Add reporting dashboards for admin and management

Phase 5: Mobile or Web Portal for Students

- Roll out a student-facing portal or mobile app for:
    - Checking plans
    - Tracking attendance
    - Viewing diet logs
    - Managing memberships

Successful deployment will centralize all gym operations and drastically improve transparency, efficiency, and user satisfaction at SAU.

11.3 Limitations of the Implementation

Despite being functional, the system has the following limitations:

1. Basic User Interface

The current web app is simple and lacks advanced UX/UI design.

2. No Real-Time Features

Features such as live workout tracking or biometric attendance are not integrated.

3. Limited Error Handling

The system relies heavily on valid input and minimal validation.

4. No Multi-Device Support

The current implementation works only on the local machine (localhost) and not on mobile devices.

5. No Authentication Levels in Front-End

Although MySQL has role-based privileges, the front-end login system does not fully enforce different role-based dashboards.

6. JSON Querying Limitations

Exercise routines stored in JSON are not deeply queryable without advanced SQL functions.

11.4 Future Scopes

The system can be enhanced in several ways to provide a complete professional-grade solution:

1. Full Web Portal / Mobile App

Develop Android/iOS or responsive web dashboards for:

- Members
- Trainers
- Gym administrators

2. Enhanced Analytics

Add dashboards for:

- Monthly revenue
- Attendance heatmaps
- Trainer performance metrics

3. Biometric or RFID Attendance

Use RFID cards or fingerprint scanners to record entry automatically.

4. Integration With University Systems

Link with:

- SAU student database
- University payment portal
- Campus ID system

12. BIBLIOGRAPHY AND REFERENCES

Books & Academic References

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th Edition). McGraw-Hill Education.

Online Documentation

4. MySQL Documentation. "MySQL 8.0 Reference Manual." https://dev.mysql.com/doc/
5. phpMyAdmin Official Documentation. https://docs.phpmyadmin.net/
6. PHP Manual – MySQLi Functions. https://www.php.net/manual/en/book.mysqli.php

Tutorials and Technical Resources

7. W3Schools. "SQL Tutorial." https://www.w3schools.com/sql/
8. W3Schools. "PHP MySQL Database Tutorial." https://www.w3schools.com/php/php_mysql_intro.asp
9. GeeksforGeeks. "ER Model, Keys, Normalization and DBMS Concepts." https://www.geeksforgeeks.org/dbms/
10. StackOverflow Community Discussions (Database Schema, MySQL Triggers, Stored Procedures). https://stackoverflow.com/

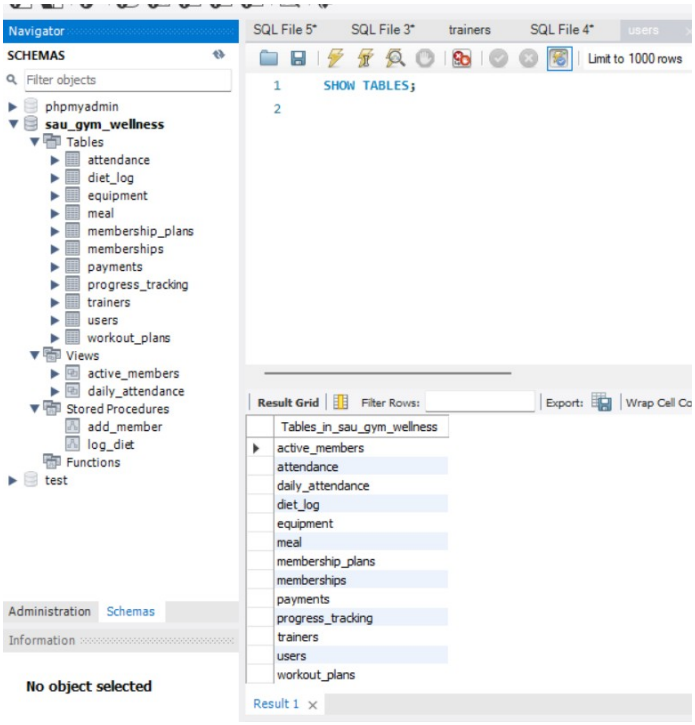
Project-Specific References

11. XAMPP Documentation – Apache & MySQL Local Server Setup. https://www.apachefriends.org/docs/
12. South Asian University – Campus Facilities and Gym Information (for contextual understanding). https://www.sau.int/

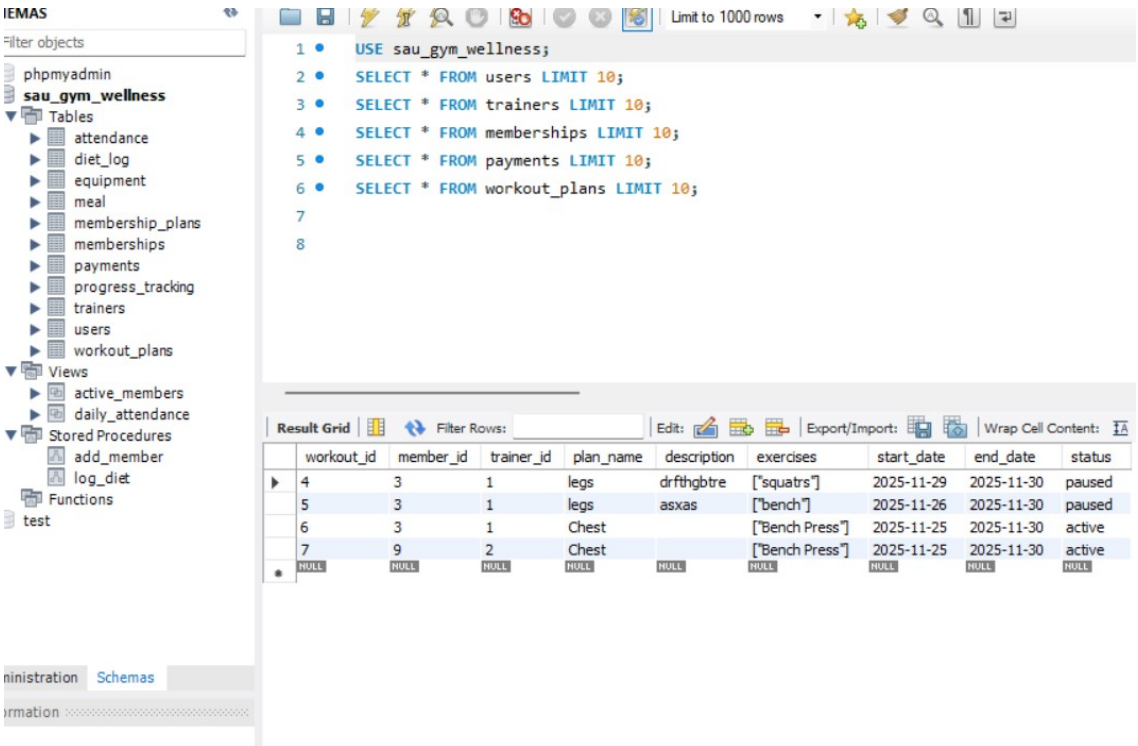
Additional Tools Used

13. MySQL Workbench User Guide. https://dev.mysql.com/doc/workbench/en/
14. Visual Studio Code – Editor Documentation (used for PHP/HTML pages). https://code.visualstudio.com/docs

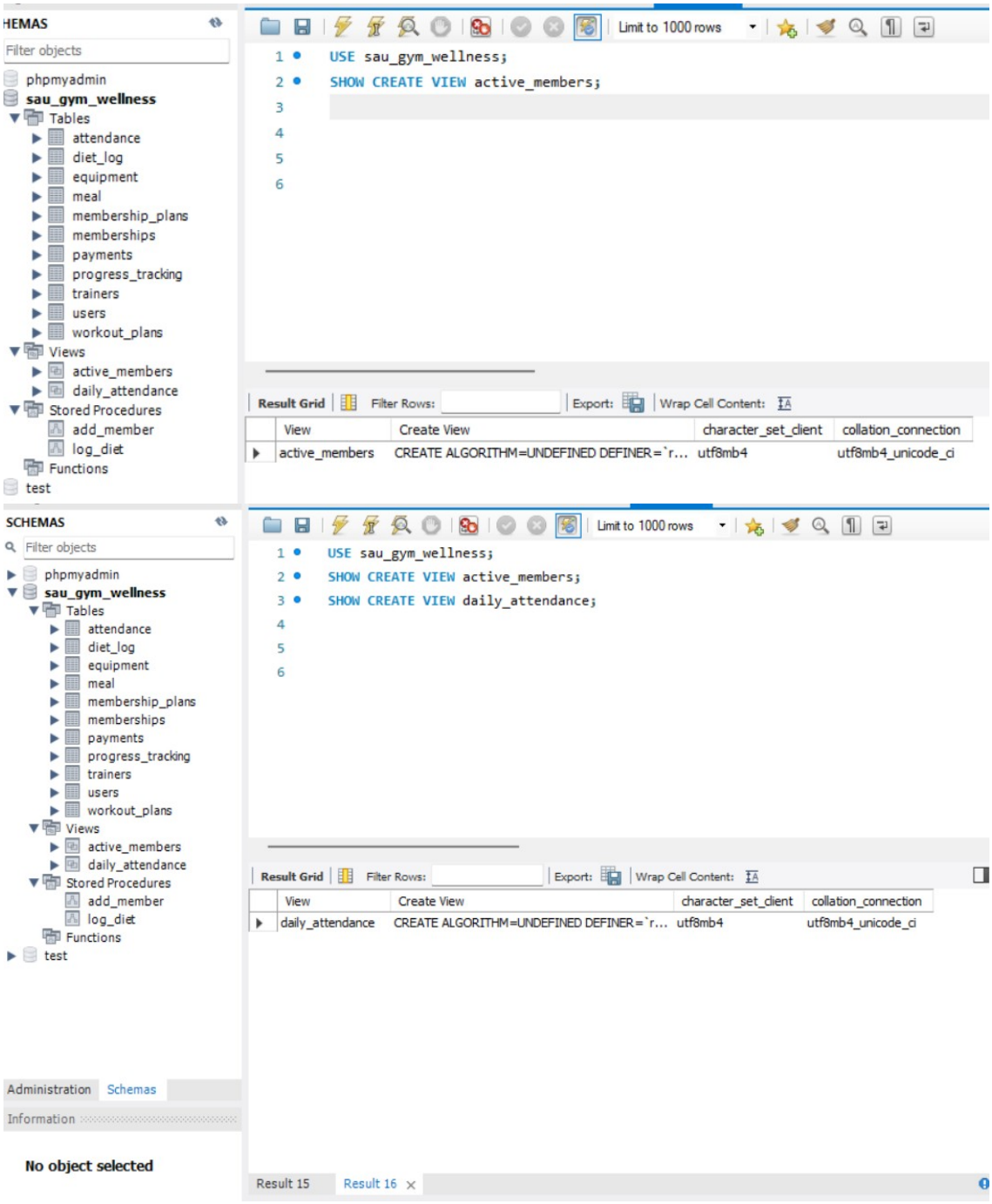
13. APPENDIX – Screenshots of Implementation



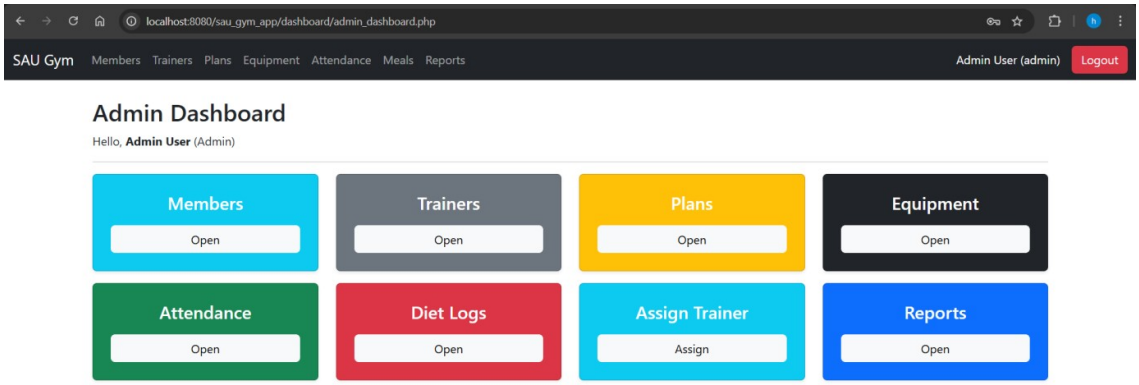
DB tables

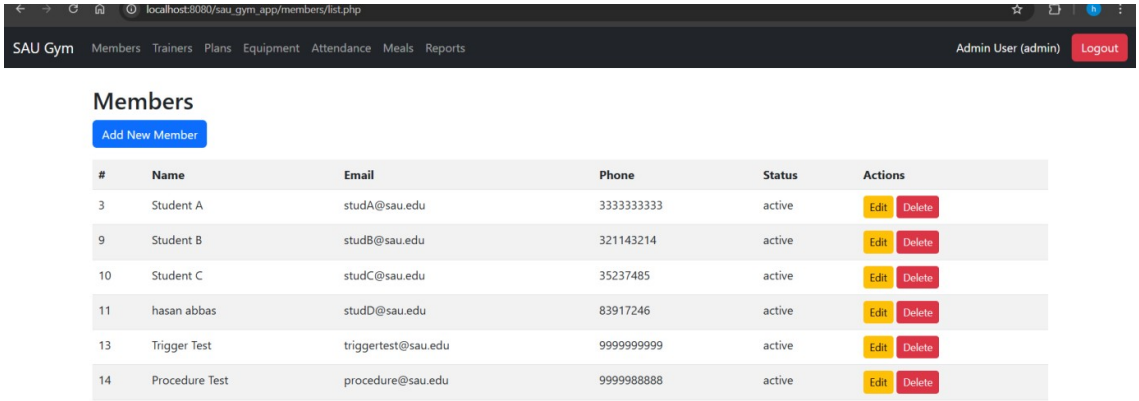


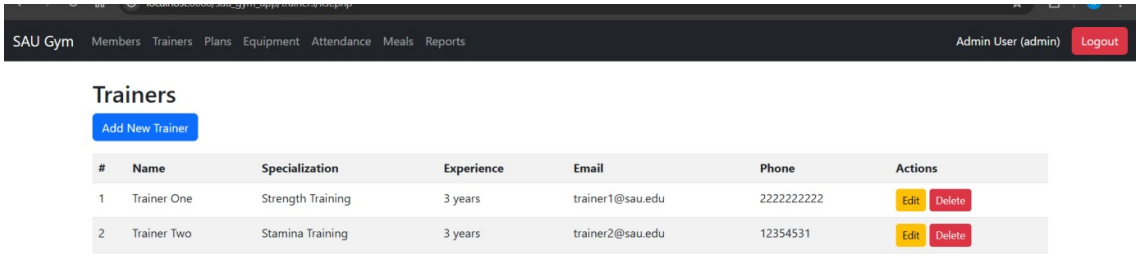
Views

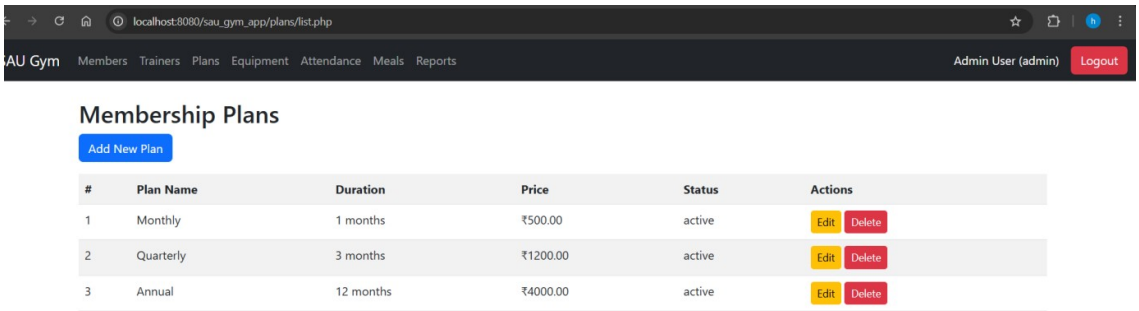

HEMAS
Filter objects
phpmyadmin
sau_gym_wellness
Tables
attendance
diet_log
equipment
meal
membership_plans
memberships
payments
progress_tracking
trainers
users
workout_plans
Views
active_members
daily_attendance
Stored Procedures
add_member
log_diet
Functions
test
Limit to 1000 rows
1 USE sau_gym_wellness;
2 SHOW CREATE VIEW active_members;
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
View
Create View
character_set_client
collation_connection
active_members
CREATE ALGORITHM=UNDEFINED DEFINER=`r...
utf8mb4
utf8mb4_unicode_ci
SCHEMAS
Filter objects
phpmyadmin
sau_gym_wellness
Tables
attendance
diet_log
equipment
meal
membership_plans
memberships
payments
progress_tracking
trainers
users
workout_plans
Views
active_members
daily_attendance
Stored Procedures
add_member
log_diet
Functions
test
Limit to 1000 rows
1 USE sau_gym_wellness;
2 SHOW CREATE VIEW active_members;
3 SHOW CREATE VIEW daily_attendance;
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
View
Create View
character_set_client
collation_connection
daily_attendance
CREATE ALGORITHM=UNDEFINED DEFINER=`r...
utf8mb4
utf8mb4_unicode_ci
Administration
Schemas
Information
No object selected
Result 15
Result 16

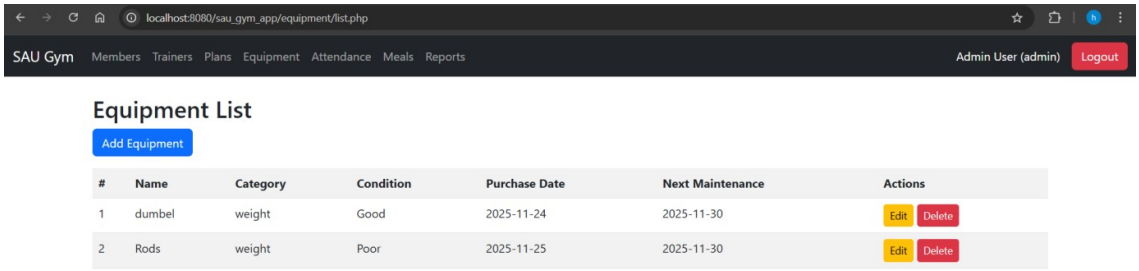
ADMIN DASHBOARD WORKING

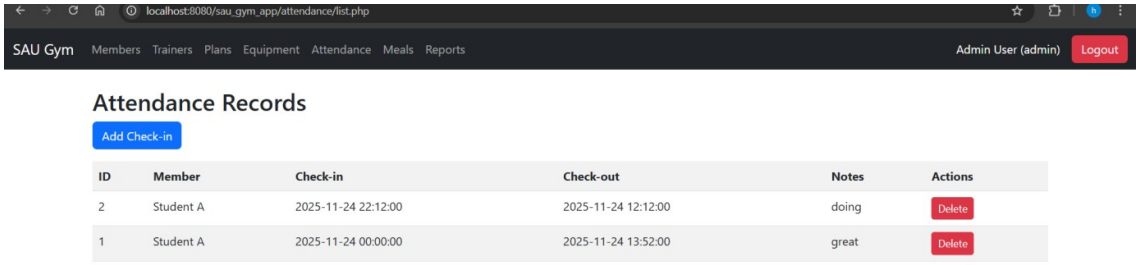


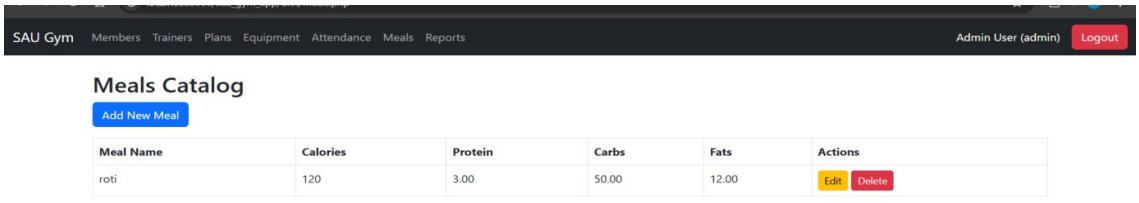


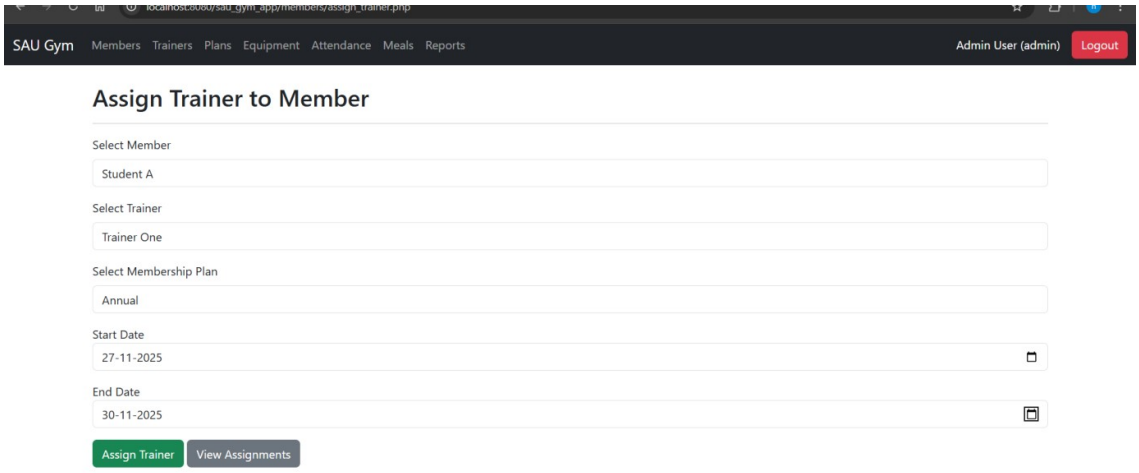


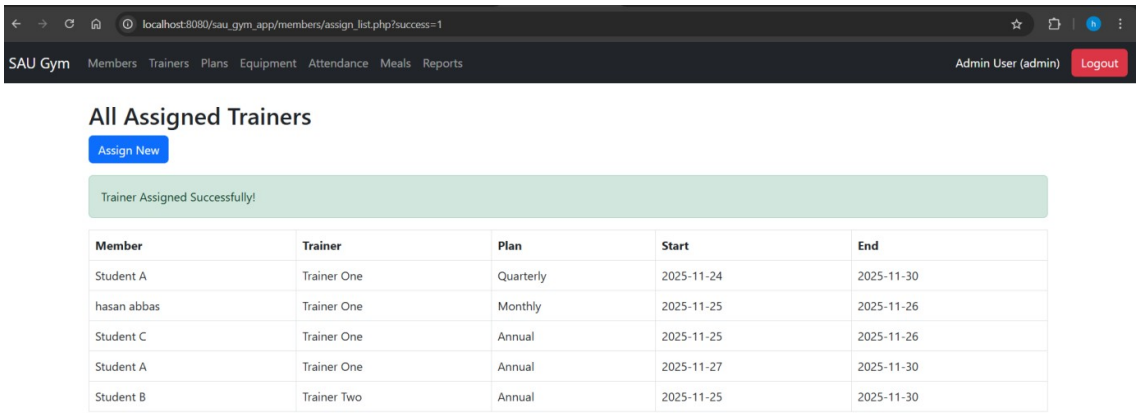

localhost:8080/sau_gym_app/plans/list.php
AU Gym
Members Trainers Plans Equipment Attendance Meals Reports
Admin User (admin)
Logout
Membership Plans
Add New Plan
Plan Name Duration Price Status Actions
1 Monthly 1 months ₹500.00 active Edit Delete
2 Quarterly 3 months ₹1200.00 active Edit Delete
3 Annual 12 months ₹4000.00 active Edit Delete

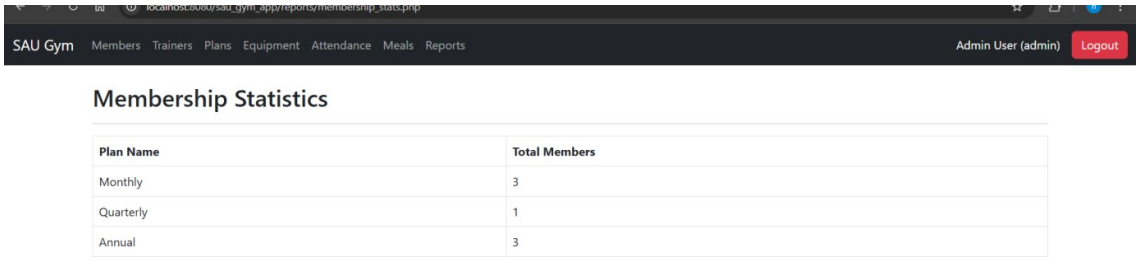

localhost:8080/sau_gym_app/equipment/list.php
SAU Gym
Members Trainers Plans Equipment Attendance Meals Reports
Admin User (admin)
Logout
Equipment List
Add Equipment
Name Category Condition Purchase Date Next Maintenance Actions
1 dumbel weight Good 2025-11-24 2025-11-30 Edit Delete
2 Rods weight Poor 2025-11-25 2025-11-30 Edit Delete

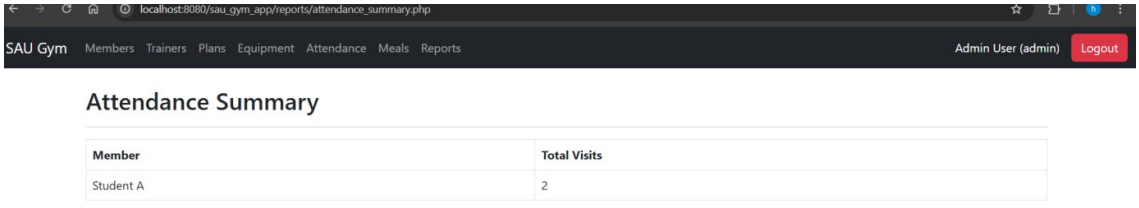

localhost:8080/sau_gym_app/attendance/list.php
SAU Gym
Members Trainers Plans Equipment Attendance Meals Reports
Admin User (admin)
Logout
Attendance Records
Add Check-in
ID Member Check-in Check-out Notes Actions
2 Student A 2025-11-24 22:12:00 2025-11-24 12:12:00 doing Delete
1 Student A 2025-11-24 00:00:00 2025-11-24 13:52:00 great Delete

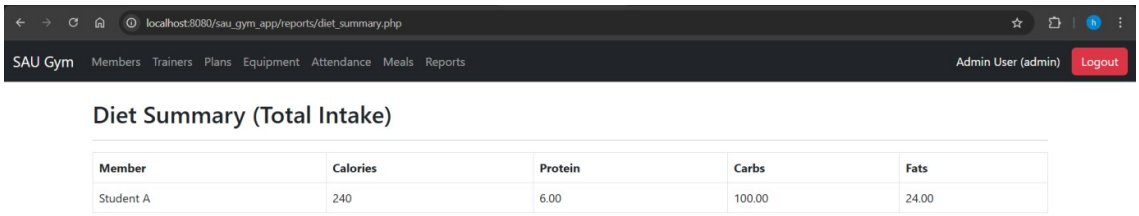

SAU Gym
Members Trainers Plans Equipment Attendance Meals Reports
Admin User (admin)
Logout
Meals Catalog
Add New Meal
Meal Name Calories Protein Carbs Fats Actions
roti 120 3.00 50.00 12.00 Edit Delete

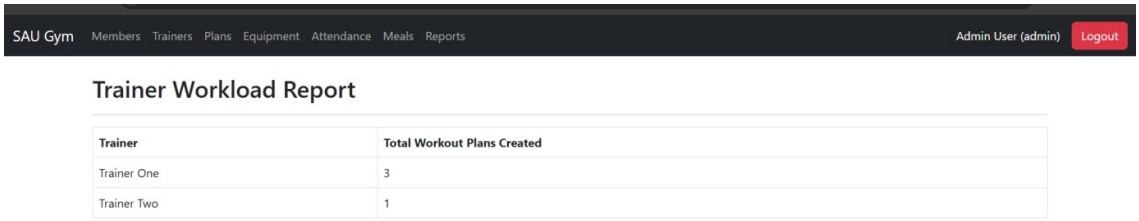

SAU Gym
Members
Trainers
Plans
Equipment
Attendance
Meals
Reports
Admin User (admin)
Logout
Assign Trainer to Member
Select Member
Student A
Select Trainer
Trainer One
Select Membership Plan
Annual
Start Date
27-11-2025
End Date
30-11-2025
Assign Trainer
View Assignments


localhost:8080/sau_gym_app/members/assign_list.php?success=1
SAU Gym
Members
Trainers
Plans
Equipment
Attendance
Meals
Reports
Admin User (admin)
Logout
All Assigned Trainers
Assign New
Trainer Assigned Successfully!
Member
Trainer
Plan
Start
End
Student A
Trainer One
Quarterly
2025-11-24
2025-11-30
hasan abbas
Trainer One
Monthly
2025-11-25
2025-11-26
Student C
Trainer One
Annual
2025-11-25
2025-11-26
Student A
Trainer One
Annual
2025-11-27
2025-11-30
Student B
Trainer Two
Annual
2025-11-25
2025-11-30

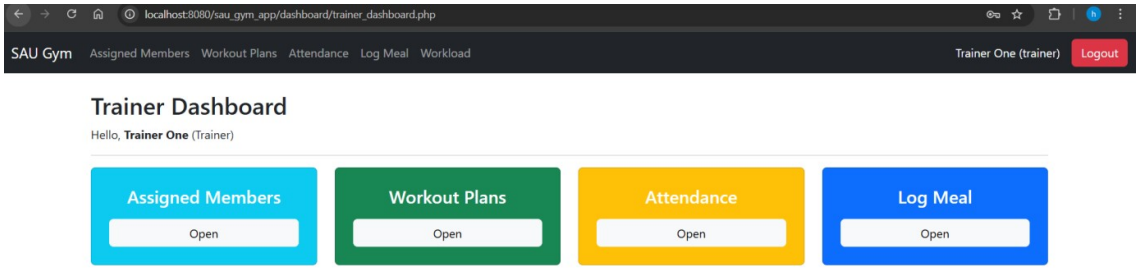

SAU Gym
Members
Trainers
Plans
Equipment
Attendance
Meals
Reports
Admin User (admin)
Logout
Membership Statistics
Plan Name
Total Members
Monthly
3
Quarterly
1
Annual
3


localhost:8080/sau_gym_app/reports/attendance_summary.php
SAU Gym
Members
Trainers
Plans
Equipment
Attendance
Meals
Reports
Admin User (admin)
Logout
Attendance Summary
Member
Total Visits
Student A
2



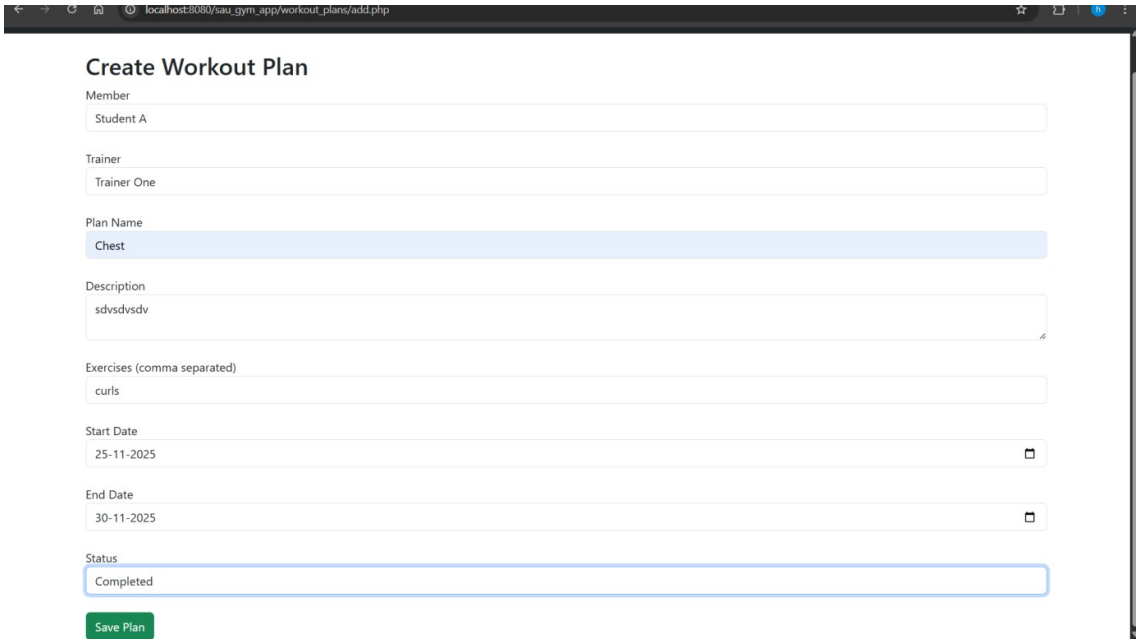


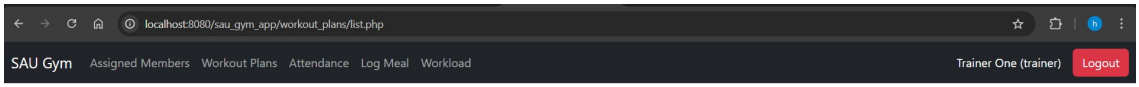
TRAINER DASHBOARD WORKING



Assigned Members

Name	Email	Phone	Membership Start	Membership End
Student A	studA@sau.edu	3333333333	2025-11-24	2025-11-30
hasan abbas	studD@sau.edu	83917246	2025-11-25	2025-11-26
Student C	studC@sau.edu	35237485	2025-11-25	2025-11-26
Student A	studA@sau.edu	3333333333	2025-11-27	2025-11-30

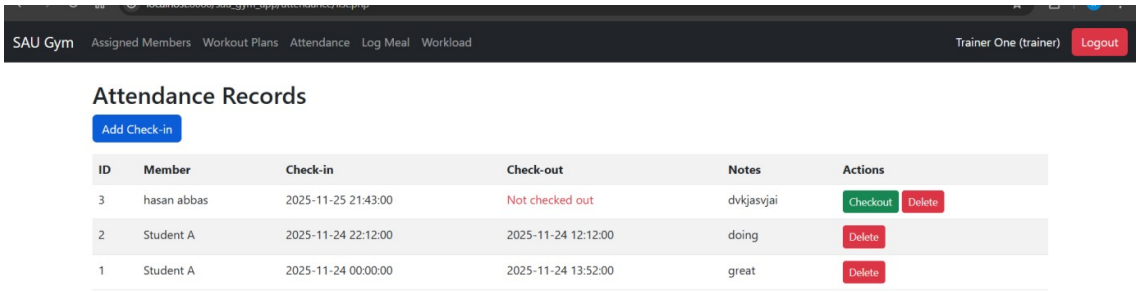


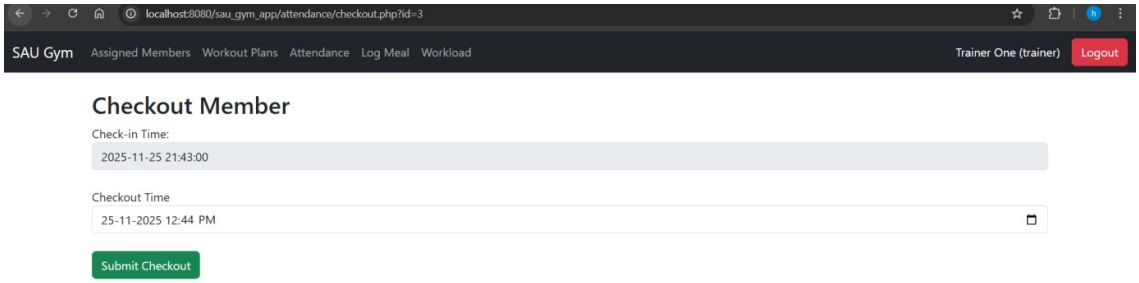


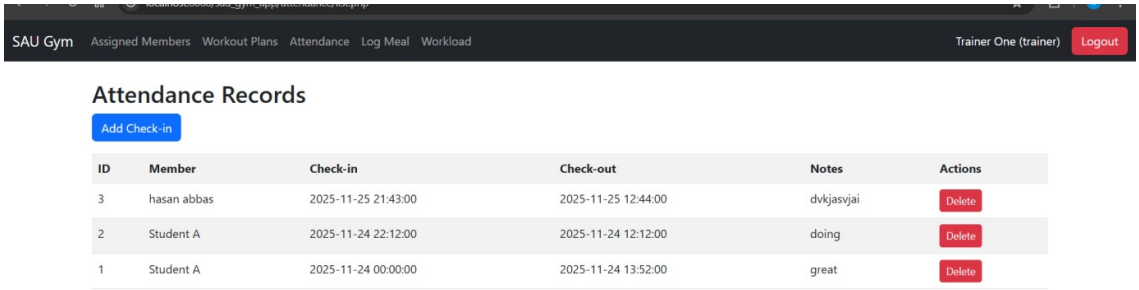
Workout Plans

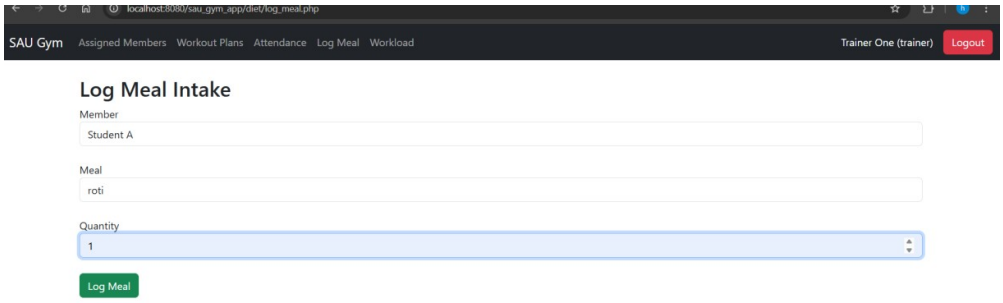
Create Workout Plan

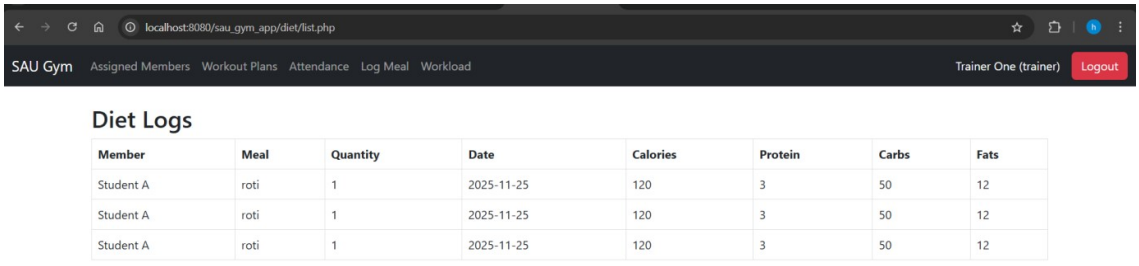
Member	Trainer	Plan Name	Duration	Status	Actions
Student A	Your Trainer	legs	2025-11-29 → 2025-11-30	Paused	View / Edit Delete
Student A	Your Trainer	legs	2025-11-26 → 2025-11-30	Paused	View / Edit Delete
Student A	Your Trainer	Chest	2025-11-25 → 2025-11-30	Active	View / Edit Delete
Student A	Your Trainer	Chest	2025-11-25 → 2025-11-30	Completed	View / Edit Delete


SAU Gym
Assigned Members
Workout Plans
Attendance
Log Meal
Workload
Trainer One (trainer)
Logout
Attendance Records
Add Check-in
ID
Member
Check-in
Check-out
Notes
Actions
3
hasan abbas
2025-11-25 21:43:00
Not checked out
dvkjasvjai
Checkout
Delete
2
Student A
2025-11-24 22:12:00
2025-11-24 12:12:00
doing
Delete
1
Student A
2025-11-24 00:00:00
2025-11-24 13:52:00
great
Delete

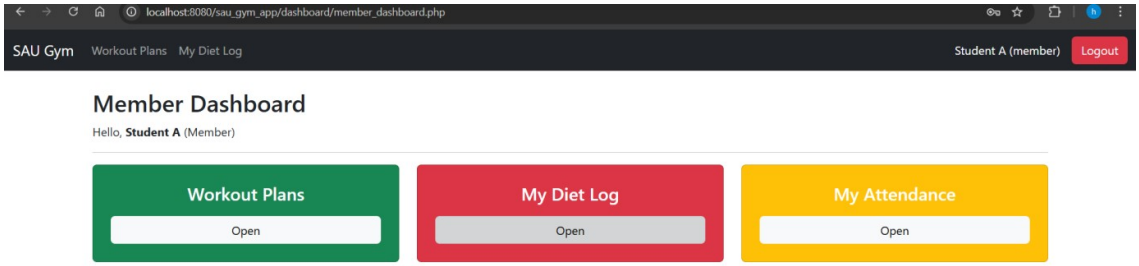

localhost:8080/sau_gym_app/attendance/checkout.php?id=3
SAU Gym
Assigned Members
Workout Plans
Attendance
Log Meal
Workload
Trainer One (trainer)
Logout
Checkout Member
Check-in Time:
2025-11-25 21:43:00
Checkout Time
25-11-2025 12:44 PM
Submit Checkout

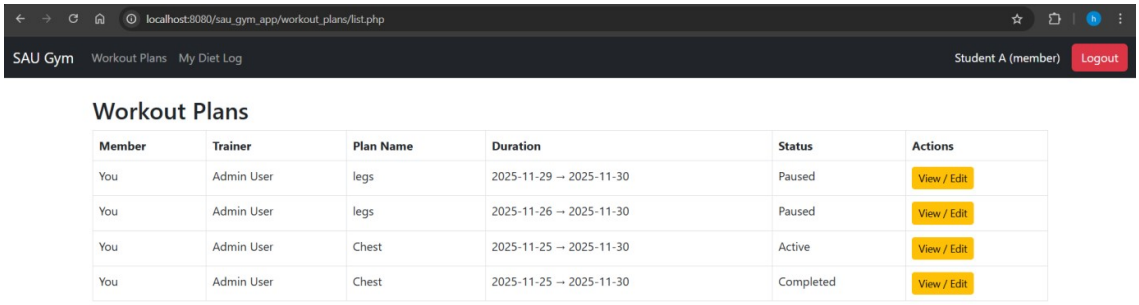

SAU Gym
Assigned Members
Workout Plans
Attendance
Log Meal
Workload
Trainer One (trainer)
Logout
Attendance Records
Add Check-in
ID
Member
Check-in
Check-out
Notes
Actions
3
hasan abbas
2025-11-25 21:43:00
2025-11-25 12:44:00
dvkjasvjai
Delete
2
Student A
2025-11-24 22:12:00
2025-11-24 12:12:00
doing
Delete
1
Student A
2025-11-24 00:00:00
2025-11-24 13:52:00
great
Delete


localhost:8080/sau_gym_app/diet/log_meal.php
SAU Gym
Assigned Members
Workout Plans
Attendance
Log Meal
Workload
Trainer One (trainer)
Logout
Log Meal Intake
Member
Student A
Meal
roti
Quantity
1
Log Meal

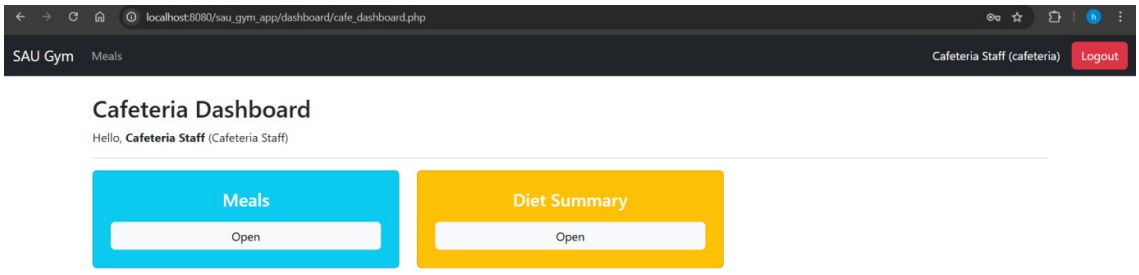

localhost:8080/sau_gym_app/diet/list.php
SAU Gym
Assigned Members
Workout Plans
Attendance
Log Meal
Workload
Trainer One (trainer)
Logout
Diet Logs
Member
Meal
Quantity
Date
Calories
Protein
Carbs
Fats
Student A
roti
1
2025-11-25
120
3
50
12
Student A
roti
1
2025-11-25
120
3
50
12
Student A
roti
1
2025-11-25
120
3
50
12

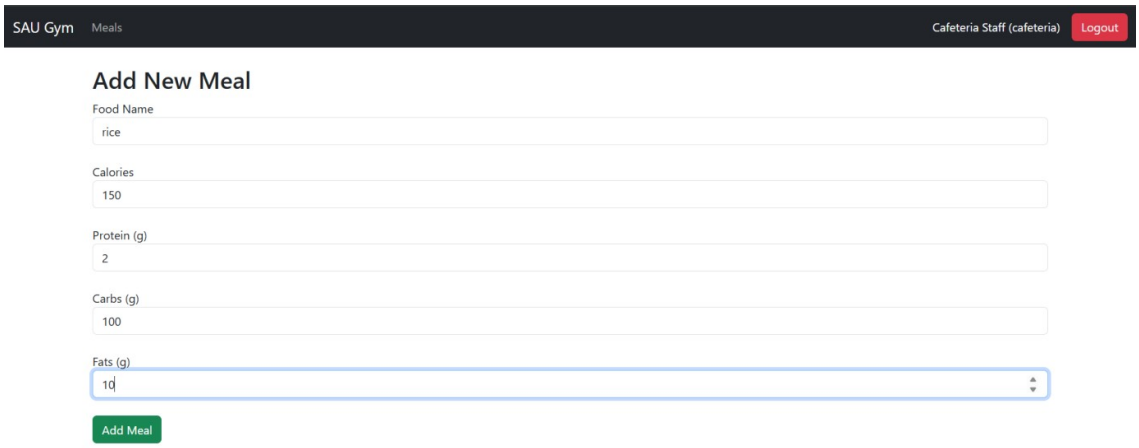
MEMBER DASHBOARD WORKING

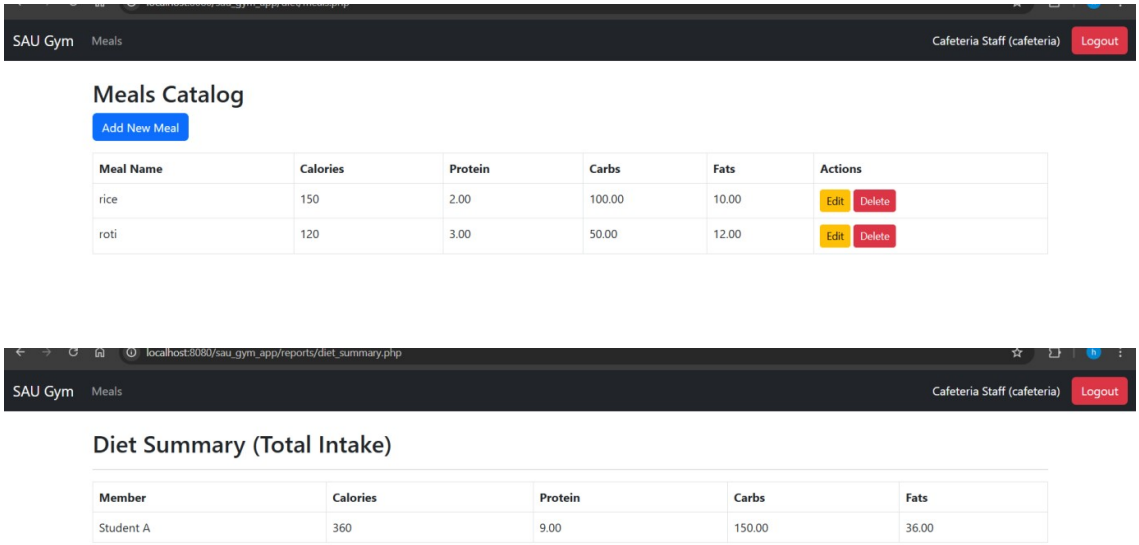




CANTEEN DASHBOARD WORKING







GIT HUB LINK FOR FRONT END

https://github.com/MayNotBeHasan/Gym-Management-System